

DEEP STUDY

Docker DKR.02

نصب Docker، نسخه‌ها، پلتفرم‌ها و روش‌های توزیع

DS-DKR-02	شناسه سند
1.0.0	نسخه
canonical-study	وضعیت
DevOps / Docker	مسیر
2026-08-12	Research cutoff
Core	سطح

این سند دومین واحد Core مسیر Docker Mastery است. هدف آن ساختن توانایی واقعی برای انتخاب deployment مناسب، نصب قابل اتکا، تشخیص componentها و نسخه‌ها، و اتصال امن CLI به daemon درست است؛ نه صرفاً اجرای یک installer.

فهرست مطالب

۶	راهنمای استفاده از این درس
۶	زمان و ریزدانشی این واحد
۶	Scope Contract
۶	مشخصات سند
۷	داخل Scope
۷	خارج از Scope
۸	پیش‌نیازها
۸	بعد از تسلط به این سند هنوز چه چیزی بلد نیستی؟
۸	هدف‌های یادگیری
۹	۱. اول مدل نصب را درست کن: «Docker» یک فایل اجرایی واحد نیست
۹	۱.۱ client-only هم ممکن است
۹	۱.۲ «نصب شد» با «قابل استفاده است» فرق دارد
۱۰	۲. Docker Engine، Docker CLI و Docker Desktop را از هم جدا نگه دار
۱۰	۲.۱ Docker Engine روی Linux
۱۰	۲.۲ Docker Desktop
۱۱	۲.۳ جدول انتخاب اولیه
۱۱	۳. Editions و نام‌گذاری: معماری را با licensing قاطی نکن
۱۲	۴. Distribution mechanism: Docker چگونه به سیستم تو می‌رسد؟
۱۲	۴.۱ روش پیشنهادی: repository رسمی + package manager
۱۲	۴.۲ package file دستی
۱۲	۴.۳ convenience script
۱۲	۴.۴ static binaries
۱۳	۵. Stable، Test، version pinning و مفهوم «latest»
۱۳	۵.۱ «latest» را از search snippet بگیر
۱۳	۵.۲ چرا pinning مهم است؟
۱۳	۵.۳ upgrade با «همه package‌ها»
۱۴	۶. نصب رسمی روی Ubuntu VM: مسیر مرجع Lab
۱۴	۶.۱ baseline قبل از نصب
۱۴	۶.۲ conflict‌ها را بشناس

۱۴	 repository رسمی را اضافه کن ۶.۳
۱۵	 package set را نصب کن ۶.۴
۱۵	 service را verify کن ۶.۵
۱۵	 ۷. Unix socket و privilege: چرا permission denied فقط یک annoyance نیست
۱۶	 ۷.۱ سه انتخاب access
۱۶	 ۸. Rootful در برابر Rootless: deployment shape را ببین
۱۶	 Rootful ۸.۱
۱۶	 Rootless ۸.۲
۱۷	 ۸.۳ اشتباه تشخیصی رایج
۱۷	 ۹. Platform reality: Linux، macOS و Windows یک deployment نیستند
۱۷	 Linux Engine native ۹.۱
۱۷	 Linux روی Docker Desktop ۹.۲
۱۸	 macOS ۹.۳
۱۸	 Windows ۹.۴
۱۸	 ۱۰. Docker Context: آدرس daemon را به عنوان state قابل نام گذاری مدیریت کن
۱۸	 ۱۰.۱ چرا context بهتر از export پراکنده است؟
۱۹	 ۱۰.۲ precedence را بشناس
۱۹	 ۱۱. DOCKER_HOST : ابزار مفید، منبع state پنهان
۱۹	 ۱۲. Remote daemon: اول SSH، بعد TLS؛ TCP plain را default نکن
۲۰	 SSH روی Context ۱۲.۱
۲۰	 ۱۲.۲ چرا SSH خوب است؟
۲۰	 TCP روی TLS ۱۲.۳
۲۰	 ۱۲.۴ TCP 2375 بدون TLS: legacy/insecure و در remote address حذف شده
۲۱	 ۱۳. Client/Server/API version: سه version را با هم یکی نگیر
۲۱	 ۱۳.۱ negotiation چه می کند؟
۲۱	 ۱۳.۲ DOCKER_API_VERSION برای debug است، نه default دائمی
۲۱	 ۱۴. Version inventory کامل: «docker --version» کافی نیست
۲۲	 ۱۴.۱ چرا بعضی command ها وجود ندارند؟
۲۲	 ۱۵. Package inventory: نسخه binary را به package provenance وصل کن
۲۳	 ۱۶. Current baseline 29.7.2: چه چیزهایی در این درس مهم اند؟
۲۳	 ۱۷. Installation decision tree
۲۳	 ۱۷.۱ سؤال اول: daemon کجا باید اجرا شود؟

۲۳	 ۱۷.۲ سؤال دوم: update چه کسی را مدیریت می‌کند؟
۲۴	 ۱۷.۳ سؤال سوم: failure domain چیست؟
۲۴	 ۱۸. Failure Mode ۱: docker: command not found
۲۴	 ۱۹. Failure Mode ۲: Cannot connect to the Docker daemon
۲۵	 ۲۰. Failure Mode ۳: permission denied روی docker.sock
۲۵	 ۲۱. Failure Mode ۴: context اشتباه و «containerهای گم‌شده»
۲۶	 ۲۲. Failure Mode ۵: client/daemon mismatch
۲۶	 ۲۳. Failure Mode ۶: package conflict یا split provenance
۲۶	 ۲۴. Failure Mode ۷: remote TCP نامن
۲۷	 ۲۵. Debugging Playbook یک صفحه‌ای
۲۷	 ۲۶. Production guidance: نصب قابل اتکا چه شکلی است؟
۲۸	 ۲۷. Lab اصلی A: نصب package-managed روی Linux VM
۲۸	 ۲۷.۱ پیش‌نیاز و ایمنی
۲۸	 ۲۷.۲ قبل از اجرا پیش‌بینی کن
۲۸	 ۲۷.۳ نصب
۲۹	 ۲۷.۴ observation بعد از نصب
۲۹	 ۲۷.۵ smoke container
۲۹	 ۲۸. Lab اصلی B: context برای daemon remote با SSH
۲۹	 ۲۸.۱ topology
۲۹	 ۲۸.۲ آماده‌سازی remote
۳۰	 ۲۸.۳ context بساز
۳۰	 ۲۸.۴ بدون تغییر remote، default را query کن
۳۰	 ۲۸.۵ evidence تفاوت daemonها
۳۰	 ۲۸.۶ cleanup
۳۱	 ۲۹. Lab C: failure drillهای عمدی
۳۱	 ۲۹.۱ API override نامعتبر
۳۱	 ۲۹.۲ wrong context
۳۱	 ۲۹.۳ socket permission
۳۱	 ۲۹.۴ VM disposable در daemon down
۳۱	 ۳۰. Lab اختیاری D: Rootless deployment shape
۳۲	 ۳۱. گزارش Lab: evidence حداقلی

۳۳	۳۲. برداشت‌های اشتباه رایج
۳۳	۳۲.۱ «اگر --version docker جواب داد، Docker سالم است»
۳۳	۳۲.۲ «Docker Desktop همان Docker Engine با UI است»
۳۳	۳۲.۳ «عضویت docker group یعنی non-root Docker»
۳۳	۳۲.۴ «port 2375 فقط یک port مدیریتی است»
۳۳	۳۲.۵ «version Engine = version containerd = version runc»
۳۳	۳۲.۶ «context فقط یک shortcut برای host string است»
۳۳	۳۲.۷ «reinstall اولین قدم debugging است»

۳۳. سؤال‌های پیش‌بینی و چالش

۳۴	۳۳. Interview Ladder
۳۴	۳۴.۱ پاسخ ۳۰ ثانیه‌ای: Docker را روی Linux چطور نصب می‌کنی؟
۳۵	۳۴.۲ پاسخ حدود ۳ دقیقه: Engine و Desktop چه فرقی دارند؟
۳۵	۳۴.۳ Deep dive: docker version چه چیزی درباره compatibility می‌گوید؟
۳۵	۳۴.۴ Deep dive: چرا docker group security-sensitive است؟

۳۵. تمرین دونفره Attack / Defend

۳۶. سؤال‌های Teach-back

۳۷. Mastery Checklist

۳۶	فهم مفهومی
۳۶	عملیات
۳۷	Failure و Debugging
۳۷	ارتباط و مصاحبه

۳۸. تمرین خواندن Source / Docs

۳۹. سؤال‌های حل‌نشده محیط تو

۴۰. جمع‌بندی یک صفحه‌ای

۴۱. قدم بعدی

منابع

۴۲. Changelog

راهنمای استفاده از این درس

این درس دومین واحد Core در مسیر Docker Mastery است. در DKR.01 مدل ذهنی کانتینر را ساختیم؛ اینجا قرار نیست دوباره تعریف کانتینر را باز کنیم. هدف این واحد این است که بتوانی یک محیط Docker را درست انتخاب، نصب، شناسایی، نسخه‌بندی و به daemon درست متصل کنی و وقتی نصب یا اتصال خراب شد، از روی evidence بفهمی مشکل در کدام لایه است.

این درس را مثل یک راهنمای «کپی و اجرا» بخوان. برای هر command اول پیش‌بینی کن چه چیزی باید ببینی، بعد اجرا کن و خروجی واقعی محیط خودت را در Lab Journal ثبت کن. هر جا نسخه، platform، package یا default مطرح است، خروجی واقعی سیستم تو از مثال این سند مهم‌تر است.

قاعده این واحد: تا وقتی نتوانی جواب بدهی «docker CLI الان دقیقاً به کدام daemon، با چه context و چه API version وصل است؟»، هنوز نصب Docker را واقعاً نفهمیده‌ای.

زمان و ریزدانی این واحد

- زمان پیشنهادی مطالعه فعال: حدود ۴ تا ۷ ساعت همراه با Lab و failure drill.
- outcome اصلی: ساخت و تشخیص یک Docker deployment قابل اتکا از سطح package تا client/daemon connection.
- این واحد عمده‌اً وارد معماری درونی `dockerd -> containerd -> shim -> runc` نمی‌شود؛ مالک آن DKR.03 است.
- جزئیات عمیق user namespace، rootless، و محدودیت‌های آن در DKR.34 باز می‌شود.

Scope Contract

مشخصات سند

مورد	مقدار
Document ID	DS-DKR-02
Curriculum ID	DKR.02
عنوان	نصب، Editions، Platforms و Distribution
Depth	Core
Version	1.0.0

مورد	مقدار
Research cutoff	2026-08-12
Source baseline	official Docker Docs / 1.55 API / 29.7.2 Docker Engine
Upstream	readiness META.DKR.02 ، DKR.01
Downstream	DKR.31 ، DKR.30 ، PLT.01 ، DKR.03

داخل Scope

- فرق Docker CLI، Docker Desktop، Docker Engine و daemon از دید نصب و عملیات.
- روش‌های رسمی نصب Docker Engine روی Linux و نقش package manager.
- package‌های اصلی docker-ce، docker-ce-cli، containerd.io، docker-buildx-plugin و docker-compose-plugin.
- stable/test channels، نصب نسخه مشخص، pinning و upgrade awareness.
- نصب با convenience script، package file، package repository و static binaries و trade-off هرکدام.
- تفاوت عملیاتی macOS، Linux و Windows برای اجرای Linux/Windows containers.
- rootful در برابر rootless در حد deployment shape و install/connection surface.
- Docker context، local Unix socket، DOCKER_HOST، DOCKER_CONTEXT، SSH و TLS برای daemon remote.
- تفاوت client version، server version و Engine API version negotiation.
- inventory کردن نسخه‌های containerd، BuildKit، Buildx، Compose، CLI، Engine، runc و .failure‌های نصب و اتصال: API، wrong context، socket permission، daemon down، package conflict، mismatch و remote daemon نامن.

خارج از Scope

- معماری داخلی کامل Engine و code path اجرای container؛ مالک: DKR.03.
- daemon configuration کامل و host integration؛ مالک: DKR.30.
- lifecycle و object model؛ مالک: DKR.04 و DKR.05.
- storage/network limitations و rootless internals، users mapping؛ مالک: DKR.34.
- Docker Desktop architecture عمیق و feature surface؛ مالک: PLT.01 و PLT.02.
- upgrade/deprecation playbook؛ مالک: DKR.31.

پیش‌نیازها

- **Knowledge prerequisite:** DKR.01 برای progression مطالعاتی باید mastered شود. ساخت و انتشار این PDF جای mastery آن را نمی‌گیرد.
- **Operational prerequisite:** readiness مربوط به META.DKR.02 باید قبل از Lab واقعی وجود داشته باشد: baseline، snapshot/rebuild، Linux VM disposable، ابزارهای پایه و ثبت .baseline
- آشنایی پایه با shell، sudo، package manager و SSH کمک می‌کند؛ اگر نداری، commandها را آهسته و با توضیح بخوان.

بعد از تسلط به این سند هنوز چه چیزی بلد نیستی؟

بعد از این درس باید بتوانی محیط Docker را نصب و تشخیص بدهی، اما هنوز قرار نیست بتوانی یک request را با docker run یا containerd trace کنی، network/storage internals را توضیح بدهی یا daemon production را harden و upgrade کنی. این مرز عمده است.

هدف‌های یادگیری

بعد از mastery این واحد باید بتوانی:

۱. از روی platform و use case تصمیم‌گیری بگیری Engine native، Docker Desktop، remote Engine یا rootless Engine مناسب‌تر است.
۲. یک نصب رسمی Linux را با package repository انجام بدهی و package‌های نصب‌شده را توضیح بدهی.
۳. نسخه را فقط با `docker --version` گزارش نکنی؛ client/server/API/runtime componentها را جدا inventory کنی.
۴. بفهمی Docker CLI با چه precedence‌ای context/host را انتخاب کرده و به endpoint درست وصل شوی.
۵. remote daemon را با SSH یا TLS امن کنی و بفهمی چرا unauthenticated TCP یک host-root boundary خطرناک بوده و در Engine‌های جدید برای remote address حذف/مسدود شده است.
۶. failure‌های رایج نصب/connection/version را evidence-driven debug کنی.
۷. برای upgrade یا production installation توضیح بدهی چرا package-managed + pinning معمولاً از static binary قابل‌اتکاتر است.

۱. اول مدل نصب را درست کن: «Docker» یک فایل اجرایی واحد نیست

وقتی کسی می‌گوید «Docker را نصب کردم»، سؤال بعدی باید این باشد: دقیقاً چه چیزهایی نصب شده‌اند و کدام process دارد سرویس می‌دهد؟

در ساده‌ترین مدل Linux، حداقل با این نقش‌ها طرف هستی:

- `docker` : client/CLI command را می‌گیرد و به Engine API درخواست می‌فرستد.
- `dockerd` : daemon اصلی Docker Engine server سمت Engine API.
- `containerd` و `runc` : runtime stack پایین‌تر که نسخه‌هایشان مستقل‌اند؛ internals در DKR.03 می‌آید.
- `docker buildx` : CLI plugin برای build surface.
- `docker compose` : CLI plugin برای Compose.
- BuildKit: engine/build backend که نسخه‌اش می‌تواند با packaging و deployment متفاوت باشد.

مدل ذهنی نصب: نصب Docker یعنی انتخاب یک **deployment shape** و یک **distribution mechanism** برای چند component؛ نه صرفاً قرار دادن binary به نام `docker` در PATH.

۱.۱ client-only هم ممکن است

روی macOS می‌توان `docker` static client را داشت، ولی static client شامل `dockerd` نیست؛ برای اجرای container باید daemon روی VM یا host remote داشته باشی. Docker Docs برای macOS و Windows client/server desktop scenarios، Docker Desktop را پیشنهاد می‌کند. [R4]

پس این command:

```
docker --version
```

فقط ثابت می‌کند client در PATH هست؛ ثابت نمی‌کند daemon سالم، local یا حتی موجود است.

۱.۲ «نصب شد» با «قابل استفاده است» فرق دارد

چهار check مستقل داشته باش:

```
command -v docker
docker --version
docker context show
docker version
```

- اولی وجود executable را نشان می‌دهد.
- دومی فقط version کلاینت را نشان می‌دهد.
- سومی context انتخاب‌شده را نشان می‌دهد.
- چهارمی تلاش می‌کند client و server را هر دو ببیند و بهترین command پایه برای تشخیص اتصال است. [R8]

۲. Docker Engine، Docker Desktop و Docker CLI را از هم جدا نگه دار

۲.۱ Docker Engine روی Linux

Docker Engine روی Linux می‌تواند native روی kernel همان host اجرا شود. روش رسمی Docker برای distribution‌های پشتیبانی‌شده، استفاده از package repository مخصوص همان distribution است. Docker Docs در صفحه نصب Engine، platform‌های پشتیبانی‌شده و stable/test channel را جدا می‌کند. [R1]

در Ubuntu package set رسمی معمولاً این‌هاست: [R2]

```
docker-ce
docker-ce-cli
containerd.io
docker-buildx-plugin
docker-compose-plugin
```

این نام‌ها مهم‌اند چون نشان می‌دهند Engine، CLI، container runtime bundle و CLI plugin‌ها یک package واحد نیستند.

۲.۲ Docker Desktop

Docker Desktop یک product با integration‌های VM/backend، UI، host، و component‌های bundled است. روی Linux هم Docker Desktop برخلاف Engine native، یک VM اجرا می‌کند و context مخصوص `desktop-linux` می‌سازد؛ به همین دلیل image/container‌های Engine native قبلی لزوماً داخل Desktop دیده نمی‌شوند. [R12][R13]

روی macOS، Linux containerها داخل Linux VM اجرا می‌شوند. Docker Desktop برای Mac چند Virtual Machine Manager دارد و انتخاب VMM وابسته به معماری host و نسخه product است. [R15]

روی Windows، Docker Desktop می‌تواند backend مبتنی بر WSL 2 یا Hyper-V داشته باشد؛ مسیر دقیق به edition و installation mode ویندوز وابسته است. [R14][R16]

۲.۳ جدول انتخاب اولیه

وضعیت	انتخاب معمول	دلیل اصلی	نکته مهم
Linux server/VM	-Docker Engine package managed	native service و automation ساده	access به socket را جدی بگیر
Linux developer desktop	Docker Desktop یا Engine	Engine سبک‌تر؛ Desktop integration بیشتر	Desktop Linux VM/context جدا دارد
macOS developer	remote یا Docker Desktop Engine	native Linux daemon روی macOS وجود ندارد	CLI-only به daemon نیاز دارد
برای Windows developer Linux containers	Docker Desktop	WSL 2/Hyper-V backend	host و /guest filesystem network semantics مهم‌اند
native برای Windows Windows containers	Docker Desktop مناسب یا Windows Engine scenario	Windows kernel/container mode	با Linux container یکی نیست
CI/remote workstation	local CLI + remote Engine	جداسازی client از daemon	SSH/TLS و context ضروری است

۳. Editions و نام‌گذاری: معماری را با licensing قاطی نکن

واژه‌هایی مثل Desktop subscription tier، Community، یا نام‌های تاریخی CE/EE ممکن است در طول زمان عوض شوند. در این مسیر، اول از **component identity** حرف می‌زنیم و بعد از product/licensing.

Docker Docs خود Engine را در installation docs با عنوان Docker CE / Docker Engine معرفی می‌کند و Engine را open-source project مرتبط با Moby توضیح می‌دهد؛ در مقابل Docker Desktop product پشتیبانی‌شده Docker است و شرایط subscription جدا دارد. [R1]

خطای رایج: از روی عبارت «Community» یا «Desktop» درباره runtime architecture نتیجه‌گیری نکن. برای architecture باید ببینی daemon کجاست، روی چه kernelی اجرا می‌شود، endpoint چیست و component‌های واقعی چه نسخه‌ای دارند.

برای licensing یا شرایط تجاری همیشه همان روز صفحه رسمی را دوباره بخوان. این سند عمداً policy تجاری را به‌عنوان قانون دائمی حفظ نمی‌کند؛ چون version-sensitive و قراردادی است.

۴. Distribution mechanism: Docker چگونه به سیستم تو می‌رسد؟

۴.۱ روش پیشنهادی: repository رسمی + package manager

برای Linux production-like environment، روش package-managed معمولاً بهترین default است چون:

- dependency و service unit ها یکپارچه‌تر مدیریت می‌شوند؛
- upgrade/downgrade مسیر مشخص‌تری دارد؛
- package inventory قابل audit است؛
- security update process می‌تواند با host policy هماهنگ شود.

Docker Docs نصب از repository رسمی را برای distribution های پشتیبانی‌شده مستند کرده است. [R1][R2]

۴.۲ package file دستی

اگر host به repository دسترسی ندارد، می‌توان `.deb` یا `.rpm` های مشخص را دانلود و نصب کرد. مزیت: artifact دقیق و قابل انتقال. عیب: هر upgrade نیاز به مدیریت دستی package های جدید دارد. [R2]

۴.۳ convenience script

`get.docker.com` برای bootstrap و test مفید است، اما برای محیطی که version control و change review مهم است باید محتاط باشی. command script را قبل از اجرا دانلود و بخوان؛ و بعد از نصب، package inventory را ثبت کن. [R2]

```
curl -fsSL https://get.docker.com -o get-docker.sh
less get-docker.sh
sudo sh get-docker.sh
```

این الگو برای lab قابل قبول است، ولی در production نباید «curl | sh» جای deployment policy شود.

۴.۴ static binaries

Docker Docs صریحاً static binary installation را بیشتر مناسب testing می‌داند و برای production توصیه نمی‌کند؛ چون update امنیتی خودکار package manager را نداری، upgrade دستی‌تر است و static binary ممکن است تمام functionality بسته‌های dynamic را نداشته باشد. [R4]

همچنین packaging component ها می‌تواند با package-managed installation فرق کند. در baseline Engine 29.7.0، release notes برای static binaries containerd v2.3.3 و runc v1.4.3 را گزارش کرده است؛ 29.7.2 نیز BuildKit را به v0.32.2 به‌روزرسانی کرده است. این اعداد را به همه installation ها تعمیم نده. [R5]

قاعده نسخه component: release notes می‌گوید یک release چه چیزی را بسته‌بندی کرده؛ اما Source of Truth محیط تو خروجی binary/package واقعی همان host است.

۵. Stable، Test، version pinning و مفهوم «latest»

Docker Engine دو update channel اصلی stable و test دارد. stable برای release عمومی و test برای pre-release است؛ test ممکن است featureهای early-access و breaking change داشته باشد. [R1]

۵.۱ «latest» را از search snippet بگیر

در research این سند صفحه رسمی release notes شاخه 29 مستقیماً باز شد. در تاریخ 2026-08-12 بالاترین release ثبت‌شده 29.7.2 با تاریخ 2026-08-05 است. [R5]

این baseline برای فهم سند است، نه دستور کور برای lab. هنگام اجرای lab:

```
docker version
apt-cache madison docker-ce 2>/dev/null || true
dnf list --showduplicates docker-ce 2>/dev/null || true
```

اول availability repository خودت را ببین.

۵.۲ چرا pinning مهم است؟

اگر امروز latest نصب کنی و یک ماه بعد lab را تکرار کنی، ممکن است behavior یا dependency متفاوت شود. برای reproducibility، حداقل این‌ها را ثبت کن:

```
OS / distribution / release
kernel
Docker CLI version
Docker Engine version
Engine API max/min
containerd version
runc version
BuildKit / buildx / compose version
installation method
package versions
```

۵.۳ upgrade با «همه packageها»

چون Buildx و Compose plugin جدا package می‌شوند، upgrade فقط docker-ce می‌تواند inventory ناهمگون بسازد. دستور upgrade را از installation page همان distribution بگیر و package set را یکجا review کن. [R2]

۶. نصب رسمی روی Ubuntu VM: مسیر مرجع Lab

این بخش برای یک Ubuntu VM disposable نوشته شده. اگر Debian/RHEL/Fedora داری، منطق یکسان است اما command repository را از صفحه رسمی همان distribution بگیر.

قبل از شروع: snapshot بگیر. این service، lab package و network behavior host را تغییر می‌دهد. روی workstation حساس یا production اجرا نکن.

۶.۱ baseline قبل از نصب

```
set -u
printf '=== OS ===\n'
cat /etc/os-release
printf '\n=== kernel ===\n'
uname -a
printf '\n=== cgroup ===\n'
stat -fc %T /sys/fs/cgroup
printf '\n=== docker before ===\n'
command -v docker || true
command -v dockerd || true
systemctl status docker --no-pager 2>/dev/null || true
```

در journal بنویس چه چیزی از قبل وجود داشت. «ماشین تمیز» را فرض نکن.

۶.۲ conflict ها را بشناس

Docker Docs برای Ubuntu package های غیررسمی/متعارض مثل `docker.io`، `docker-compose`، `podman-docker` و `package` های مستقل `runc / containerd` را قبل از نصب رسمی مطرح می‌کند، چون Docker Engine package خودش dependency runtime را با `containerd.io` مدیریت می‌کند. [R2]

قبل از حذف، inventory بگیر:

```
dpkg -l | grep -E 'docker|containerd|runc|podman' || true
```

در lab disposable می‌توانی command رسمی `uninstall conflict` ها را طبق docs اجرا کنی. در production هرگز package را فقط چون اسمش شبیه conflict است بدون review impact حذف نکن.

۶.۳ repository رسمی را اضافه کن

برای command دقیق `keyring/source` همان روز صفحه رسمی Ubuntu را دنبال کن. علت مهم‌تر از syntax است:

- `keyring` تعیین می‌کند package metadata از source مورد اعتماد `verify` شود.

- architecture, source file distribution codename, stable channel را مشخص می‌کند.
- `apt update` inventory نسخه‌های قابل نصب را refresh می‌کند.

بعد بررسی کن:

```
apt-cache policy docker-ce docker-ce-cli containerd.io
```

۶.۴ package set را نصب کن

الگوی رسمی فعلی [R2]: Ubuntu

```
sudo apt install \
docker-ce \
docker-ce-cli \
containerd.io \
docker-buildx-plugin \
docker-compose-plugin
```

اگر می‌خواهی نسخه مشخص نصب کنی، ابتدا نسخه‌های repository را list کن و version string واقعی را انتخاب کن؛
version string را از این PDF کپی نکن.

۶.۵ service را verify کن

```
systemctl is-enabled docker || true
systemctl is-active docker || true
systemctl status docker --no-pager
sudo docker run --rm hello-world
```

hello-world فقط smoke test است. ثابت نمی‌کند security posture، version inventory، context یا remote access درست است.

۷. Unix socket و privilege: چرا permission denied فقط یک annoyance نیست

در Engine، daemon rootful معمولاً با privilege بالا اجرا می‌شود و local CLI از Unix socket مانند `/var/run/docker.sock` استفاده می‌کند. Docker Docs هشدار می‌دهد membership در `docker` group در `root-level privileges` می‌دهد. [R3]

```
ls -l /var/run/docker.sock  
id  
getent group docker || true
```

اگر این خطا را دیدی:

```
permission denied while trying to connect to the Docker daemon socket
```

اول ownership/membership و context را بررسی کن؛ socket را با `chmod 666` «حل» نکن.

۷.۱ سه انتخاب access

۱. `sudo docker ...` : ساده برای lab، ولی فایل‌های `~/docker` ممکن است بعداً owner اشتباه بگیرند.

۲. افزودن user به `docker group` : راحت، ولی عملاً privilege سطح root به آن user می‌دهد. [R3]

۳. `rootless mode: daemon` و `container` ها را بدون `host-root` اجرا می‌کند؛ trade-off های خودش را دارد. [R6]

anti-pattern: `sudo chmod 666 /var/run/docker.sock` دسترسی daemon را برای هر local user باز می‌کند. این یک fix امنیتی نیست.

۸. Rootful در برابر Rootless: deployment shape را ببین

۸.۱ Rootful

مدل معمول package-managed Engine:

```
user -> docker CLI -> /var/run/docker.sock -> rootful dockerd -> runtime stack
```

service معمولاً system-wide است:

```
sudo systemctl status docker
```

۸.۲ Rootless

در `rootless mode` خود daemon و `container` ها داخل `user namespace` و بدون `host-root` اجرا می‌شوند. Docker Docs آن را از `usersns-remap` جدا می‌کند: در `usersns-remap` daemon هنوز `rootful` است، اما در `rootless` خود daemon هم `non-root` است. [R6]

پیش‌نیازهای مهم شامل `newgidmap / newuidmap` و `subordinate UID/GID range` است. `-setup tool package` managed فعلی context به نام `rootless` می‌سازد و `user service` ایجاد می‌کند. [R6]

```
dockerd-rootless-setuptool.sh install
systemctl --user status docker
docker context show
docker info
```

socket نمونه `rootless`:

```
unix:///run/user/<uid>/docker.sock
```

۸.۳ اشتباه تشخیصی رایج

ممکن است هم `rootful Engine` و هم `rootless Engine` روی یک `host` داشته باشیم. اگر `docker ps` «containerها» را گم کرده» نشان می‌دهد، قبل از هر چیز `context` و `endpoint` را چک کن:

```
docker context ls
docker context inspect "$(docker context show)"
```

۹. Platform reality: Linux، macOS و Windows یک deployment نیستند

۹.۱ Linux Engine native

در `Linux Engine native`، `daemon` و `Linux container`ها از `Linux host` kernel استفاده می‌کنند. این ساده‌ترین `boundary` برای `Lab`های `internals` آینده است.

۹.۲ Docker Desktop روی Linux

`Docker Desktop for Linux` یک `VM` اجرا می‌کند و `context desktop-linux` دارد. بنابراین `filesystem`، `network` و `image store` آن را با `host native Engine` یکی فرض نکن. [R12][R13]

```
docker context ls
```

اگر هم `Engine native` و هم `Desktop` نصب باشند، ممکن است حداقل `default` و `desktop-linux` را ببینی.

macOS ۹.۳

Linux kernel، macOS kernel، Linux container نیست؛ Docker Desktop برای Linux VM اجرا می‌شوند. static docker binary در macOS client-only است و daemon/runtime environment نمی‌آورد. [R15][R4]

نتیجه عملی: مسیر network، file sharing و performance بین macOS host و Linux VM اهمیت دارد؛ این جزئیات در PLT.01 می‌آید.

Windows ۹.۴

روی Windows، Docker Desktop برای Linux containers معمولاً از WSL 2 یا Hyper-V backend استفاده می‌کند. WSL 2 یک Linux kernel دارد. Docker Desktop همچنین scenarioهای Windows containers را دارد که isolation و host compatibility متفاوت است. [R16][R14]

static Windows Engine binaries docker.exe / dockerd.exe برای native Windows containers هستند؛ آنها روی Linux daemon Windows kernel نمی‌سازند. [R4]

۱۰. Docker Context: آدرس daemon را به عنوان state قابل

نام‌گذاری مدیریت کن

یک context ترکیبی از name، endpoint config و TLS information است. default context معمولاً Unix socket local را هدف می‌گیرد. [R7]

```
docker context ls
docker context show
docker context inspect default
```

۱۰.۱ چرا context بهتر از export پراکنده است؟

با context می‌توانی endpoint را name بدهی، inspect/export/import کنی و اشتباه «این shell به کدام daemon وصل است؟» را کاهش بدهی.

مثال local:

```
docker context create local-explicit \
--docker host=unix:///var/run/docker.sock
```

۱۰.۲ precedence را بشناس

Docker CLI docs می‌گوید context انتخاب‌شده می‌تواند با environment variable یا global flag override شود. به‌ویژه DOCKER_CONTEXT از DOCKER_HOST precedence بالاتری دارد. [R9][R7]

برای debug:

```
printf 'DOCKER_CONTEXT=%s\n' "${DOCKER_CONTEXT-}"
printf 'DOCKER_HOST=%s\n' "${DOCKER_HOST-}"
docker context show
docker context ls
docker version
```

اگر command رفتار عجیبی دارد، فقط به ستاره context list نگاه نکن؛ environment variableها ممکن است override کرده باشند.

۱۱. DOCKER_HOST: ابزار مفید، منبع state پنهان

DOCKER_HOST endpoint را برای client مشخص می‌کند. نمونه‌ها:

```
export DOCKER_HOST=unix:///var/run/docker.sock
export DOCKER_HOST=ssh://docker-user@host1.example.com
```

برای یک command موقت:

```
DOCKER_HOST=ssh://docker-user@host1.example.com docker info
```

مشکل operational این است که CI variable، shell profile، یا IDE ممکن است آن را پنهانی set کرده باشد. برای همین contextهای named معمولاً auditability بهتری می‌دهند.

۱۲. Remote daemon: اول SSH، بعد TLS؛ TCP plain را default

نکن

Docker daemon به‌طور پیش‌فرض از local Unix socket استفاده می‌کند. می‌توان آن را remote کرد، اما remote daemon یعنی remote control سطح بسیار بالا روی Docker Docs. host هشدار می‌دهد هر endpoint شبکه‌ای Engine API یک رمز امنیتی حساس است. در نسخه‌های جدید، unauthenticated TCP روی آدرس دیگر یک deployment پشتیبانی‌شده نیست؛ برای remote access از SSH یا TLS verification استفاده کن. [R18][R11][R10]

۱۲.۱ SSH روی Context

روش کم‌اصطکاک برای lab دوماشینه:

```
docker context create lab-remote \  
--docker host=ssh://docker-user@REMOTE_HOST \  
--description="DKR.02 remote lab engine"  
  
docker --context lab-remote version  
docker --context lab-remote info
```

remote user باید روی آن host permission دسترسی به Docker socket داشته باشد. [R10]

۱۲.۲ چرا SSH خوب است؟

- authentication و host identity روی SSH می‌آید.
- لازم نیست daemon را روی TCP عمومی expose کنی.
- SSH policy با key management موجود سازمان قابل ترکیب است.

برای performance می‌توان SSH connection reuse را در `~/.ssh/config` فعال کرد؛ ControlMaster/ docs؛ ControlPersist را پیشنهاد می‌کند. [R10]

۱۲.۳ TLS روی TCP

اگر API باید روی TCP قابل دسترس باشد، mTLS با CA/server/client certificate یکی از راه‌های رسمی است. port مرسوم secure Docker TLS برابر 2376 است. client possession کلید client را مثل root credential ببین. [R10]

۱۲.۴ TCP 2375 بدون TLS: legacy/insecure و در remote address حذف‌شده

port 2375 به‌طور تاریخی با Docker TCP بدون TLS شناخته می‌شد و هنوز ممکن است در مثال‌های قدیمی، wrapper یا بعضی product setting‌های خاص ببینی. اما Engine policy deprecation می‌گوید **unauthenticated TCP connections** از v26 deprecated شدند و target removal آن‌ها v28 بود؛ از v27 به بعد اگر daemon برای remote TCP تنظیم شده باشد، `--tls=false` یا `--tlsverify=false` باعث startup failure می‌شود. استثنای policy برای `tcp://localhost` به معنی امن بودن exposure شبکه‌ای نیست. [R18]

پس در Engine 29 این الگو را «روش نصب remote» آموزش نمی‌دهیم؛ آن را فقط به‌عنوان **config legacy/insecure** باید تشخیص داده شود می‌شناسیم. برای remote واقعی از SSH یا TLS verification استفاده کن. [R10][R18]

Failure security drill: برای اثبات خطر، daemon را واقعاً روی `0.0.0.0:2375` باز نکن. کافی است config قدیمی را روی کاغذ/فایل نمونه تحلیل کنی و توضیح بدهی چرا Engine API historically unauthenticated به کنترل privileged host منجر می‌شد و چرا Engine جدید remote TCP را به TLS verification وادار می‌کند.

۱۳. Client/Server/API version: سه version را با هم یکی نگیر

`docker version` دو بخش Client و Server دارد. Docker CLI و daemon لازم نیست دقیقاً یک version باشند؛ آنها Engine API version negotiation انجام می‌دهند. [R17][R8]

در baseline فعلی:

```
Docker Engine 29.7.2
maximum API: 1.55
minimum API: 1.40
```

این snapshot از API reference رسمی در research cutoff است. [R17]

۱۳.۱ negotiation چه می‌کند؟

client و server بالاترین API version مشترک را انتخاب می‌کنند. اگر client جدیدتر باشد، ممکن است featureهایی داشته باشد که daemon قدیمی نمی‌شناسد؛ اگر client قدیمی‌تر باشد، featureهای جدید daemon را نمی‌بیند. compatibility best-effort است. [R17]

۱۳.۲ DOCKER_API_VERSION برای debug است، نه default دائمی

این variable negotiation را disable می‌کند. [R17][R8]

```
DOCKER_API_VERSION=1.40 docker version
```

برای failure drill روی Engine 29.7.x می‌توانی نسخه خارج از range را موقت امتحان کنی و error را ثبت کنی؛ بعد variable را unset کن.

```
export DOCKER_API_VERSION=1.39
docker version || true
unset DOCKER_API_VERSION
```

هدف حفظ error message نیست؛ هدف فهم API negotiated boundary است.

۱۴. Version inventory کامل: «docker --version» کافی نیست

یک snapshot خوب حداقل این command را اجرا می‌کند:

```
printf '=== docker CLI ===\n'
docker --version

printf '\n=== client/server/API ===\n'
docker version

printf '\n=== engine info ===\n'
docker info

printf '\n=== compose ===\n'
docker compose version 2>/dev/null || true

printf '\n=== buildx ===\n'
docker buildx version 2>/dev/null || true

printf '\n=== daemon ===\n'
dockerd --version 2>/dev/null || true

printf '\n=== containerd ===\n'
containerd --version 2>/dev/null || true

printf '\n=== runc ===\n'
runc --version 2>/dev/null || true

printf '\n=== buildctl ===\n'
buildctl --version 2>/dev/null || true
```

۱۴.۱ چرا بعضی command ها وجود ندارند؟

- Desktop ممکن است component را داخل VM/bundle نگه دارد و host shell binary مستقیم نداشته باشد.
 - Engine package-managed ممکن است BuildKit را embedded/integrated به شکلی expose کند که `buildctl` جدا نصب نباشد.
 - static client روی macOS ممکن است فقط `docker` داشته باشد و Compose/Buildx plugin را نداشته باشد.
- [R4]

پس «command not found» برای component جانبی همیشه به معنی Engine خراب نیست؛ deployment shape را در نظر بگیر.

۱۵. Package inventory: نسخه binary را به package

provenance وصل کن

روی Debian/Ubuntu:

```
dpkg -l | grep -E 'docker-ce|docker-buildx|docker-compose|containerd.io'
apt-cache policy docker-ce docker-ce-cli containerd.io
```

روی RPM-based:

```
rpm -qa | grep -E 'docker|containerd'
dnf info docker-ce docker-ce-cli containerd.io
```

این evidence به تو می‌گوید package از کدام repository آمده و چه version ی نصب است. برای incident واقعی، این اطلاعات از UI screenshot مفیدتر است.

۱۶. Current baseline 29.7.2: چه چیزهایی در این درس مهم‌اند؟

در research cutoff این سند، Docker Engine 29.7.2 آخرین release شاخه 29 در release notes رسمی است. [R5] برای DKR.02 سه نکته مهم است:

۱. **version snapshot** را ثبت کن: API Engine 29.7.x 1.55 دارد؛ minimum فعلی 1.40 است. [R17]

۲. **packaging** را با **runtime version** یکی نگیر: release 29.7.0 برای static binaries containerd v2.3.3 و runc v1.4.3 را ثبت کرده، ولی package distribution می‌تواند dependency دیگری داشته باشد. [R5]

۳. **fresh-install behavior** می‌تواند با **upgrade** فرق کند: Engine 29.0 برای fresh install image containerd store را default کرده، با exception‌هایی مثل users-remap. جزئیات storage در DKR.22 است، اما در اینجا باید بفهمی install history می‌تواند runtime behavior را تغییر دهد. [R5]

۱۷. Installation decision tree

۱۷.۱ سؤال اول: daemon کجا باید اجرا شود؟

- Linux host همان ماشین؟ -> Engine native گزینه طبیعی.
- macOS/Windows workstation؟ -> Docker Desktop یا remote Linux Engine.
- Linux desktop با Desktop features؟ -> Docker Desktop، ولی VM/context جدا را بپذیر.
- محیط بدون host-root؟ -> rootless را ارزیابی کن.
- workstation فقط client است؟ -> remote context با SSH/TLS.

۱۷.۲ سؤال دوم: update چه کسی را مدیریت می‌کند؟

- OS/package policy؟ -> official package repository + pinning.

- provisioning tool یا exact package artifacts <- ؟immutable image/CI bootstrap
- convenience script <- ؟test disposable
- maintenance burden واضح acceptance با static binary <- ؟unsupported/test platform

۱۷.۳ سؤال سوم: failure domain چیست؟

اگر Desktop VM خراب شود با host Engine فرق دارد. اگر remote daemon down شود، local CLI سالم است. اگر context اشتباه باشد، containerها «گم» نشده‌اند؛ client به daemon دیگری نگاه می‌کند.

۱.۱۸ Failure Mode: docker: command not found

مسیر debug:

```
command -v docker || true
echo "$PATH"
ls -l /usr/bin/docker /usr/local/bin/docker 2>/dev/null || true
```

سؤال‌ها:

- client package نصب شده؟
 - static binary در PATH است؟
 - Desktop CLI integration فعال است؟
 - داخل shell/WSL/VM متفاوتی هستی؟
- این failure هنوز هیچ چیز درباره daemon نمی‌گوید.

۱.۱۹ Failure Mode: Cannot connect to the Docker daemon

اول endpoint را پیدا کن:

```
env | grep '^DOCKER_' || true
docker context show
docker context inspect "$(docker context show)"
```

بعد اگر local rootful است:

```
systemctl status docker --no-pager
ls -l /var/run/docker.sock
```

اگر rootless است:

```
systemctl --user status docker --no-pager  
ls -l "$XDG_RUNTIME_DIR/docker.sock" 2>/dev/null || true
```

اگر remote است:

```
ssh docker-user@REMOTE_HOST true  
docker --context lab-remote version
```

۲۰. Failure Mode ۳: permission denied روی docker.sock

Evidence جمع کن:

```
id  
ls -ln /var/run/docker.sock  
getent group docker || true
```

تشخیص‌های ممکن:

- user عضو group نیست یا session group membership refresh نشده.
- socket owner/group غیرمنتظره است.
- context در واقع rootless/remote نیست و به rootful local اشاره می‌کند.
- shell قبلاً با sudo فایل‌های ~/.docker را root-owned کرده است. [R3]

راه‌حل را متناسب با policy انتخاب کن؛ chmod 666 default نیست.

۲۱. Failure Mode ۴: context اشتباه و «containerهای گم‌شده»

این sequence را حفظ کن:

```
docker context ls  
docker context show  
env | grep '^DOCKER_'  
docker version --format '{{.json .}}' | head -c 500
```

اگر Desktop Linux و Engine native هم‌زمان داری، default و desktop-linux image store جدا دارند. [R12]

۲۲. Failure Mode ۵: client/daemon mismatch

نشانه‌ها:

- client command option شناخته می‌شود ولی daemon endpoint feature را ندارد.
- API version error می‌بینی.
- output `docker version` client و server release متفاوت نشان می‌دهد.
- اول negotiation را ببین و `DOCKER_API_VERSION` override را حذف کن:

```
unset DOCKER_API_VERSION
docker version
```

اگر mismatch عمدی نیست، package/client provenance را هم بررسی کن؛ ممکن است CLI از path دیگری آمده باشد.

۲۳. Failure Mode ۶: package conflict یا split provenance

مثال خطرناک:

```
/usr/bin/docker از package رسمی Docker
containerd دیگر از distribution repository
runc در /usr/local/bin دستی در
compose plugin در ~/docker/cli-plugins نسخه قدیمی در
```

هرکدام ممکن است به تنهایی کار کند، اما incident را سخت می‌کند. inventory binary path + package owner بگیر:

```
command -v docker dockerd containerd runc
readlink -f "$(command -v docker)"
dpkg -S "$(command -v docker)" 2>/dev/null || true
```

۲۴. Failure Mode ۷: remote TCP ناامن

اگر config قدیمی یا automation چیزی شبیه این دارد:

```
-H tcp://0.0.0.0:2375
```

و TLS verification وجود ندارد، آن را **legacy/insecure configuration** و **migration blocker** فرض کن. در Engine 29 انتظار نداشته باش چنین remote listener بدون TLS یک configuration سالم و قابل اتکا باشد. [R18]

```
systemctl cat docker | grep -E -- '-H|ExecStart' || true
sudo grep -R '2375|hosts' /etc/docker /etc/systemd/system/docker.service.d 2>/dev/null || true
ss -lntp | grep -E ':2375|:2376' || true
```

بدون نیاز، port را باز نکن.

۲۵. Debugging Playbook یک صفحه ای

وقتی Docker «کار نمی‌کند» این ترتیب را برو:

۱. **client وجود دارد؟** `command -v docker` .
۲. **client version چیست؟** `docker --version` .
۳. **endpoint انتخابی چیست؟** `docker context show` + `env | grep '^DOCKER_'` .
۴. **client به server می‌رسد؟** `docker version` .
۵. **اگر local است service/socket سالم است؟** `systemctl` + `ls -l socket` .
۶. **اگر remote است transport سالم است؟** SSH/TLS/network .
۷. **API range چیست؟** `docker version` ; override را check کن.
۸. **provenance چیست؟** `package inventory` + `binary path` .
۹. **deployment shape چیست؟** rootful/rootless/Desktop/native/remote .
۱۰. **بعد از evidence mutation کن.** قبل از reinstall کورکورانه، علت را پیدا کن.

۲۶. Production guidance: نصب قابل اتکا چه شکلی است؟

برای production-style host:

- installation method را document کن.
- repository/source و signing configuration را ثبت کن.
- version policy و pinning مشخص داشته باش.
- upgrade قبل از rollout روی host disposable تست شود.
- `docker version` و `package inventory` در change evidence ذخیره شود.
- access به socket به عنوان privileged access مدیریت شود.
- remote API بدون authentication/TLS expose نشود.
- configuration و data directory را قبل از uninstall/upgrade بشناس.

• static binary را فقط با maintenance ownership روشن انتخاب کن.

تعریف نصب **production-grade** در این سطح: فقط «daemon بالا است» نیست؛ باید provenance، version، rollback/rebuild path و endpoint، privilege model، update path معلوم باشد.

۲۷. Lab اصلی A: نصب package-managed روی Linux VM

۲۷.۱ پیش‌نیاز و ایمنی

- VM disposable با snapshot.
- اینترنت یا repository mirror معتبر.
- shell با sudo برای rootful path.
- Lab Journal با ستون‌های Prediction / Command / Observation / Explanation.

۲۷.۲ قبل از اجرا پیش‌بینی کن

قبل از هر command جواب بنویس:

۱. انتظار داری `docker --version` قبل از نصب چه کند؟
۲. بعد از نصب، owner و mode `/var/run/docker.sock` چه خواهد بود؟
۳. `docker version` و `docker --version` چه تفاوتی خواهند داشت؟
۴. آیا بدون `sudo` می‌توانی `docker ps` اجرا کنی؟ چرا؟
۵. package‌های Compose و Buildx را جدا خواهی دید یا بخشی از `docker-ce`؟

۲۷.۳ نصب

installation page رسمی distribution خودت را باز کن و repository رسمی را اضافه کن. برای Ubuntu package set فعلی همان پنج package بخش ۶.۴ است. [R2]

۲۷.۴ observation بعد از نصب

```
systemctl status docker --no-pager
ls -l /var/run/docker.sock
docker --version
sudo docker version
sudo docker info
sudo docker compose version
sudo docker buildx version
```

بعد package inventory :

```
dpkg -l | grep -E 'docker-ce|containerd.io|docker-buildx|docker-compose'
```

۲۷.۵ smoke container

```
sudo docker run --rm hello-world
```

در journal توضیح بده این command چه چیزهایی را ثابت نمی‌کند.

۲۸. Lab اصلی B: context دوم برای daemon remote با SSH

این بخش دقیقاً syllabus requirement را پوشش می‌دهد: client واحد، دو daemon/context.

۲۸.۱ topology

```
Client machine
|- context default -> local/default daemon
`- context lab-remote -> SSH -> Linux VM remote -> Docker Engine
```

اگر فقط یک VM داری، host اصلی می‌تواند client باشد و VM daemon remote. لازم نیست TCP daemon باز کنی.

۲۸.۲ آماده‌سازی remote

روی remote host:

- Engine باید سالم باشد.
- `docker-user` باید طبق policy به socket دسترسی داشته باشد.
- SSH key authentication ترجیح دارد.

از client:

```
ssh docker-user@REMOTE_HOST 'docker version'
```

۲۸.۳ context بساز

```
docker context create lab-remote \  
--docker host=ssh://docker-user@REMOTE_HOST \  
--description="DKR.02 remote lab"
```

```
docker context ls  
docker context inspect lab-remote
```

۲۸.۴ بدون تغییر remote، default را query کن

```
docker --context lab-remote version  
docker --context lab-remote info  
docker --context lab-remote run --rm hello-world
```

۲۸.۵ evidence تفاوت daemonها

روی هر context اینها را ثبت کن:

```
docker --context default version --format '{{json .Server}}'  
docker --context lab-remote version --format '{{json .Server}}'
```

اگر versionها برابرند، باز هم ثابت کن endpoint متفاوت است.

۲۸.۶ cleanup

```
docker context rm lab-remote
```

۲۹. Lab C: failure drill های عمدی

۲۹.۱ API override نامعتبر

```
export DOCKER_API_VERSION=1.39
docker version || true
unset DOCKER_API_VERSION
docker version
```

ثبت کن error به کدام boundary اشاره می‌کند و چرا unset کردن override negotiation را برمی‌گرداند. [R17]

۲۹.۲ wrong context

یک context remote بساز، روی آن container اجرا کن، بعد به default برگرد و توضیح بده چرا container در `docker ps -a` دیده نمی‌شود.

۲۹.۳ socket permission

به جای خراب کردن permission state، mode socket، permission را observe کن و در یک shell/user فاقد group membership failure را reproduce کن اگر lab environment اجازه می‌دهد. هدف bypass امنیت نیست.

۲۹.۴ VM disposable در daemon down

```
sudo systemctl stop docker
docker version || true
sudo systemctl start docker
docker version
```

قبل از stop پیش‌بینی کن client-side output چه بخشی باقی می‌ماند.

۳۰. Lab اختیاری D: Rootless deployment shape

اگر META.DKR.02 readiness و subordinate UID/GID prerequisites داری: [R6]

```
grep ^$(whoami)": /etc/subuid /etc/subgid
dockerd-rootless-setup.sh install
systemctl --user status docker
docker context ls
docker info
```

در report تفاوت این موارد را ثبت کن:

- .service manager: system vs user
- .socket path
- .context name
- .security options `docker info`
- `dockerd` process owner

جزئیات kernel/usersns را به DKR.34 بسپار.

۳۱. گزارش Lab: evidence حداقلى

یک report کوتاه بساز که شامل این‌ها باشد:

مقدار واقعی محیط	Evidence
	OS / release
	kernel
	installation method
	Docker CLI
	Docker Engine
	API max/min
	active context
	daemon endpoint
	rootful/rootless/Desktop
	containerd
	runc
	Buildx
	Compose
	package versions
	remote context result
	failure reproduced

این جدول را با خروجی خودت پر کن؛ مثال PDF evidence محسوب نمی‌شود.

۳۲. برداشتهای اشتباه رایج

۳۲.۱ «اگر `docker --version` جواب داد، Docker سالم است»

غلط. فقط client را ثابت می‌کند؛ daemon/endpoint/API هنوز نامعلوم است.

۳۲.۲ «Docker Desktop همان Docker Engine با UI است»

ناقص. Desktop deployment و VM/backend/context integration خودش را دارد؛ روی Linux حتی image store آن از Engine native جداست. [R12]

۳۲.۳ «عضویت `docker group` یعنی `non-root Docker`»

غلط. daemon rootful باقی می‌ماند و group عملاً root-level access به آن می‌دهد. rootless deployment متفاوت است. [R6][R3]

۳۲.۴ «port 2375 فقط یک port مدیریتی است»

این مدل legacy/insecure است. Engine API control سطح بالا روی host دارد و Docker unauthenticated remote از TCP را حذف کرده است؛ برای remote access از SSH یا TLS verification استفاده کن. [R18][R10]

۳۲.۵ «`version Engine = version containerd = version runc`»

غلط. component ها independent versioning دارند و packaging method مهم است.

۳۲.۶ «context فقط یک shortcut برای `host string` است»

ناقص. context یک configuration object با endpoint و TLS info است و export/import/inspect lifecycle دارد. [R7]

۳۲.۷ «reinstall اولین قدم debugging است»

غلط. ممکن است فقط context/environment variable اشتباه باشد و reinstall evidence را از بین ببرد.

۳۳. سؤال‌های پیش‌بینی و چالش

چالش ۱

روی یک Linux laptop هم Docker Engine native و هم Docker Desktop نصب است. `docker ps` ناگهان خالی است ولی application قبلی هنوز کار می‌کند. قبل از هر restart یا reinstall چه پنج evidence می‌گیری؟

چالش ۲

CLI نسخه 29.7.2 است ولی daemon remote نسخه قدیمی‌تر دارد. command جدیدی failure می‌دهد. چگونه تشخیص می‌دهی permission، transport، یا API compatibility مشکل است؟

چالش ۳

تیمی می‌گوید «برای راحتی CI، dockerd را روی 0.0.0.0:2375 باز می‌کنیم ولی security group فقط subnet داخلی را allow می‌کند». چه boundaryهایی هنوز بدون authentication باقی مانده و alternative امن چیست؟

چالش ۴

روی Ubuntu `docker-ce` از repository رسمی نصب شده ولی `runc --version` چیزی غیرمنتظره نشان می‌دهد. چه احتمالات provenance ای بررسی می‌کنی؟

چالش ۵

یک developer می‌گوید rootless نصب کرده ولی `ps` نشان می‌دهد dockerd system هنوز root است و docker context show برابر default. چه اتفاقی احتمالاً افتاده؟

چالش ۶

چرا برای reproducible lab فقط نوشتن «Docker 29» کافی نیست؟ حداقل چه version ها و چه ghost facts را ثبت می‌کنی؟

۳۴. Interview Ladder

۳۴.۱ پاسخ ۳۰ ثانیه‌ای: Docker را روی Linux چطور نصب می‌کنی؟

پاسخ خوب باید بگوید: distribution پشتیبانی‌شده را verify می‌کنم، conflict package ها را audit می‌کنم، repository رسمی Docker را اضافه می‌کنم، Engine/CLI/containerd.io/Buildx/Compose plugin را با package manager نصب می‌کنم، service و `docker version` را verify می‌کنم، access به socket را آگاهانه تنظیم می‌کنم و version/package inventory را ثبت می‌کنم. برای production version را pin و upgrade path را تست می‌کنم.

۳۴.۲ پاسخ حدود ۳ دقیقه: Engine و Desktop چه فرقی دارند؟

باید بتوانی توضیح بدهی Engine daemon/API/runtime روی Linux native قابل اجراست؛ Desktop یک product با VM/backend و host integration است. روی macOS و Linux containers و Windows به Linux environment نیاز دارند؛ Desktop این environment را فراهم می‌کند. Desktop Linux هم VM و context desktop-linux دارد. CLI مستقل از daemon است و می‌تواند local یا remote Engine را با context/SSH/TLS کنترل کند.

۳۴.۳ Deep dive: docker version چه چیزی درباره compatibility می‌گوید؟

باید Client/Server را جدا کنی، Engine/API version را توضیح بدهی، negotiation highest-common API را شرح بدهی، نقش DOCKER_API_VERSION در disable کردن negotiation را بگویی، mismatch feature availability را تحلیل کنی و بعد به package/binary provenance و context endpoint وصل کنی.

۳۴.۴ Deep dive: چرا docker group security-sensitive است؟

باید توضیح بدهی daemon rootful است و member می‌تواند Engine API operation‌هایی درخواست کند که host resource را mount، process privileged اجرا یا filesystem را تغییر دهد؛ بنابراین group membership از نظر trust تقریباً privileged access است. rootless با این مدل یکی نیست.

۳۵. تمرین دونفره Attack / Defend

نفر A یکی از statement‌ها را دفاع کند؛ نفر B با evidence و boundary آن را attack کند:

۱. «Docker Desktop و Engine برای developer فرقی ندارند.»
 ۲. «داخل شبکه خصوصی، TCP 2375 کافی امن است.»
 ۳. «اگر client جدیدتر از daemon باشد، همیشه backward compatible است.»
 ۴. «static binary برای production ساده‌تر است چون dependency ندارد.»
 ۵. «rootless یعنی تمام محدودیت‌های امنیتی container حل شده‌اند.»
 ۶. «اگر container‌ها را نمی‌بینم، daemon data از بین رفته است.»
- بعد نقش‌ها را عوض کنید. دفاع خوب باید scope و caveat داشته باشد، نه شعار.

۳۶. سؤال‌های Teach-back

بدون note برای partner توضیح بده:

۱. از لحظه‌ای که docker shell را پیدا می‌کند تا زمانی که daemon پاسخ می‌دهد چه check‌هایی رخ می‌دهد؟

۲. چرا `docker --version` و `docker version` دو سؤال متفاوت‌اند؟
۳. روی Linux چه تفاوتی بین Engine native و Desktop Linux وجود دارد؟
۴. context چگونه از اشتباه daemon target جلوگیری می‌کند و environment variable چگونه آن را override می‌کند؟
۵. docker-group access، rootless و rootful، در یک diagram مقایسه کن.
۶. package-managed و static binary را از دید security update و reproducibility مقایسه کن.
۷. برای daemon remote چرا SSH/TLS مهم است؟
۸. اگر client/server version mismatch شد، چه evidenceای قبل از upgrade می‌گیری؟

۳۷. Mastery Checklist

این checklist را فقط وقتی تیک بزن که evidence واقعی داری.

فهم مفهومی

- ☐ . می‌توانم Docker CLI، Docker Engine، Docker Desktop و runtime component را از دید /installation deployment جدا کنم.
- ☐ . می‌توانم توضیح بدهم چرا macOS client binary به تنهایی daemon نیست.
- ☐ . می‌توانم تفاوت Engine native Linux و Desktop Linux VM/context را توضیح بدهم.
- ☐ . می‌توانم docker-group access، rootless و rootful را از هم جدا کنم.
- ☐ . می‌توانم stable/test و package/static/convenience-script trade-off را توضیح بدهم.
- ☐ . می‌توانم client/server/API version را جدا کنم و negotiation را توضیح بدهم.

عملیات

- ☐ . روی VM disposable یک installation رسمی package-managed انجام داده‌ام یا equivalent آن را با evidence کامل اجرا کرده‌ام.
- ☐ . package inventory و version snapshot واقعی ذخیره کرده‌ام.
- ☐ . می‌توانم endpoint/context فعال را با command نشان بدهم.
- ☐ . context دوم برای daemon remote با SSH ساخته و verify کرده‌ام.

- . بدون تغییر command، default context را روی remote اجرا کرده‌ام.

Debugging و Failure

- . حداقل یک daemon-down failure را reproduce و از روی `docker version` / `service state` تشخیص داده‌ام.
- . یک API mismatch/override failure را reproduce کرده‌ام.
- . permission model `/var/run/docker.sock` را با evidence توضیح می‌دهم.
- . می‌توانم wrong-context scenario را بدون reinstall تشخیص بدهم.
- . می‌توانم plain remote TCP exposure را به‌عنوان security risk توضیح بدهم و SSH/TLS alternative ارائه کنم.

ارتباط و مصاحبه

- . پاسخ ۳۰ ثانیه‌ای installation را بدون note می‌دهم.
- . پاسخ ۳ دقیقه‌ای Engine vs Desktop را بدون note می‌دهم.
- . API negotiation را در deep dive توضیح می‌دهم.
- . درس را برای partner teach-back کرده‌ام و حداقل پنج boundary question را جواب داده‌ام.

Gate DKR.02: صرف نصب شدن Docker یا ساخته شدن این PDF mastery نیست. برای عبور مطالعاتی به DKR.02 باید DKR.01 mastery و evidence همین checklist/Lab واقعی ثبت شده باشد.

۳۸. تمرین خواندن Source / Docs

- این module Core است و source trace عمیق اجباری نیست؛ اما reference reading باید انجام شود:
۱. [R1] را باز کن و platform list، stable/test، upgrade note را پیدا کن.
 ۲. [R2] را باز کن و package set رسمی، conflict package ها و نصب version مشخص را پیدا کن.
 ۳. [R7] و [R9] را بخوان و precedence context/environment را با مثال خودت بنویس.
 ۴. [R10] را بخوان و SSH context و TLS certificate trust model را خلاصه کن.
 ۵. [R17] را بخوان و API matrix نسخه Engine خودت را پیدا کن.

۶. [R6] را بخوان و rootless prerequisites و socket/context ساخته شده توسط setup tool را ثبت کن.

۳۹. سؤال‌های حل نشده محیط تو

این موارد را PDF نمی‌تواند به جای host تو جواب بدهد؛ در Lab Journal پر کن:

- architecture و distribution دقیق چیست؟
- package repository امروز چه version‌هایی ارائه می‌دهد؟
- Engine واقعی تو 29.7.2 است یا release دیگری؟
- installation تو rootless، Desktop، package-managed، یا static است؟
- daemon endpoint دقیق چیست؟
- cgroup mode و firewall backend چیست؟
- component versions واقعی چیست؟
- آیا Desktop/native Engine هم‌زمان داری؟
- remote daemon policy سازمان چیست؟

۴۰. جمع‌بندی یک صفحه‌ای

اگر فقط یک صفحه از این درس در ذهن بماند:

- Docker installation یک **deployment shape** است، نه یک binary.
- docker client می‌تواند از daemon جدا باشد.
- `docker --version` فقط client را می‌گوید؛ `docker version` client/server/API را می‌بیند.
- روی Engine native، Linux و Desktop Linux یکی نیستند؛ Desktop VM/context جدا دارد.
- روی macOS برای Linux containers به Linux VM/remote daemon نیاز داری.
- روی Windows backend و container mode را صریح شناسایی کن.
- `docker` group برابر «rootless» نیست و security-sensitive است.
- `DOCKER_HOST/context` تعیین می‌کنند CLI به کدام daemon می‌رود.
- remote daemon را با SSH یا TLS امن کن؛ plain TCP را default نکن.
- client/server نسخه‌های متفاوت می‌توانند با API negotiation کار کنند، اما feature availability را verify کن.
- همیشه version snapshot و package provenance واقعی host را ثبت کن.

۴۱. قدم بعدی

بعد از اینکه خود learner DKR.01 و DKR.02 را با evidence لازم mastered کرد، next curriculum unit:

Docker Engine: CLI -> API -> dockerd -> runtime stack معماری DKR.03

در DKR.03 دقیقاً همین connection model را یک لایه پایین تر می بریم: request از CLI وارد Engine API می شود، مسئولیت dockerd را می شکنیم و جایگاه containerd/shim/runc را بدون قاطی کردن logical architecture با packaging topology trace می کنیم.

منابع

R1 - Docker Docs: Install Docker Engine

<https://docs.docker.com/engine/install/>

نقش: installation overview و platform support، stable/test channels، upgrade.

R2 - Docker Docs: Install Docker Engine on Ubuntu

<https://docs.docker.com/engine/install/ubuntu/>

نقش: conflict package ها، package-file، specific-version install، package set، repository، convenience -script paths.

R3 - Docker Docs: Linux post-installation steps

<https://docs.docker.com/engine/install/linux-postinstall/>

نقش: systemd post-install و Unix socket، docker group، root-level privilege warning.

R4 - Docker Docs: Install Docker Engine from binaries

<https://docs.docker.com/engine/install/binaries/>

نقش: macOS client-only، production warning، static-binary caveat، Windows native binaries.

R5 - Docker Engine 29 release notes

<https://docs.docker.com/engine/release-notes/29/>

نقش: fresh-install و BuildKit/containerd/runc snapshot، current baseline 29.7.2، packaging delta، changes.

R6 – Docker Docs: Rootless mode

<https://docs.docker.com/engine/security/rootless/>

نقش: socket و rootless deployment shape، prerequisites، setup tool، user service، context.

R7 – Docker Docs: Docker contexts

<https://docs.docker.com/engine/manage-resources/contexts/>

نقش: override behavior و context anatomy، endpoint/TLS، create/use/inspect/export/import.

R8 – Docker CLI reference :docker version

<https://docs.docker.com/reference/cli/docker/version/>

نقش: API negotiation و client/server component version output.

R9 – Docker CLI base command reference

<https://docs.docker.com/reference/cli/docker/>

نقش: client environment variables و DOCKER_API_VERSION ، DOCKER_CONTEXT ، DOCKER_HOST.

R10 – Docker Docs: Protect the Docker daemon socket

<https://docs.docker.com/engine/security/protect-access/>

نقش: port 2376 و SSH context، TLS mutual authentication، credential risk.

R11 – Docker Docs: Configure remote access for Docker daemon

<https://docs.docker.com/engine/daemon/remote-access/>

نقش: remote-listener configuration؛ این صفحه هنوز برای migration/context بعضی مثال‌های 2375 را نشان می‌دهد، اما security/deprecation policy جدیدتر برای unauthenticated remote TCP در R18 میناست.

R12 – Docker Docs: Install Docker Desktop on Linux

<https://docs.docker.com/desktop/setup/install/linux/>

نقش: Linux Desktop VM و desktop-linux context.

R13 – Docker Desktop for Linux FAQ

<https://docs.docker.com/desktop/troubleshoot-and-support/faqs/linuxfaqs/>

نقش: دلیل استفاده Desktop Linux از VM و تفاوت state با host Engine.

R14 – Docker Docs: Install Docker Desktop on Windows

<https://docs.docker.com/desktop/setup/install/windows-install/>

نقش: installation modes، WSL 2/Hyper-V backends، و Windows-container constraints.

R15 – Docker Docs: Virtual Machine Manager for Docker Desktop on Mac

<https://docs.docker.com/desktop/features/vmm/>

نقش: Linux VM/VMM reality روی macOS و backend choices.

R16 – Docker Docs: WSL 2 backend on Windows

<https://docs.docker.com/desktop/features/wsl/>

نقش: filesystem/resource behavior، Linux kernel backend، و WSL integration.

R17 – Docker Engine API reference

<https://docs.docker.com/reference/api/engine/>

نقش: API version matrix، 29.7 -> API 1.55 / minimum 1.40 و negotiation semantics.

R18 – Docker Docs: Deprecated Docker Engine features

<https://docs.docker.com/engine/deprecated/>

نقش: deprecation/removal policy برای unauthenticated TCP connections؛ deprecated target در v26،

removal v28 و الزام TLS verification برای remote TCP در نسخه‌های جدید.

Changelog

1.0.0 - 2026-08-12

- اولین canonical candidate برای DKR.02 بر اساس syllabus v1.1.0.
- scope نصب/rootless/rootful، remote access، contexts، distribution/platform/edition و version inventory پوشش داده شد.
- Docker Engine 29.7.2 و API 1.55 در روز research مستقیماً از documentation رسمی verify شد.
- status فعلی TCP remote unauthenticated با deprecation policy رسمی cross-check شد تا مثال legacy 2375 به اشتباه به عنوان الگوی قابل استفاده آموزش داده نشود.
- Lab package-managed Linux + remote SSH context + failure drills + interview/teach-back/mastery اضافه شد.